

## MODULE 02

# 컴퓨터의 눈: 픽셀에서 물체 인식까지

사진은 사실 숫자 행렬! 이웃 뱀셈으로 테두리를 찾고,  
움직이는 픽셀 덩어리를 세어 자동차를 카운트하는 원리

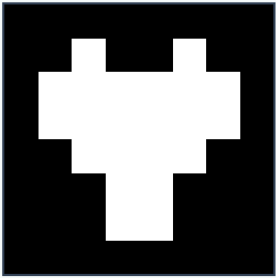
🕒 약 40분

📦 시뮬레이터 + Colab

📖 선수 지식: 뱀셈이면 충분!

아래쪽 셀과의 뱀셈 (세로 방향): [아래쪽 셀(r+1, c) - 현재 셀(r, c)] 상하 밝기가 바뀔 때 큰 숫자가 나와 '가로선(수평 테두리)'을 감지합니다.  
총합 테두리: max(오른쪽 차이, 아래쪽 차이) 세로선과 가로선이 합쳐져 완전한 물체 외곽선이 완성됩니다!  
임계값(Threshold): "차이가 몇 이상이어야 진짜 테두리로 인정할까?"를 결정하는 기준 숫자입니다.

8×8 픽셀 캔버스 (도화지) 자유기



마우스로 칸을 클릭하거나 드래그하여 흰색(255)을 칠해보세요.

☒ 0 = 검정 (어두움) ☐ 255 = 흰색 (밝음)

❤️ 하트 그리기 🚗 자동차 도면 ➡ 대각선

📄 시각 상자

💻 컴퓨터의 뇌: 8×8 숫자 행렬 (밝기 0~255) 실시간 숫자 동기화

|   |     |     |     |     |     |     |   |
|---|-----|-----|-----|-----|-----|-----|---|
| 0 | 0   | 0   | 0   | 0   | 0   | 0   | 0 |
| 0 | 0   | 255 | 0   | 0   | 255 | 0   | 0 |
| 0 | 255 | 255 | 255 | 255 | 255 | 255 | 0 |
| 0 | 255 | 255 | 255 | 255 | 255 | 255 | 0 |
| 0 | 0   | 255 | 255 | 255 | 255 | 0   | 0 |
| 0 | 0   | 0   | 255 | 255 | 0   | 0   | 0 |
| 0 | 0   | 0   | 255 | 255 | 0   | 0   | 0 |
| 0 | 0   | 0   | 0   | 0   | 0   | 0   | 0 |

🔥 실시간 뱀셈 검사기: 위 행렬 표의 칸에 마우스를 올리면 [현재 셀]과 [↖️ 오른쪽 셀], [↘️ 아래쪽 셀]의 뱀셈 계산 과정이 여기에 표시됩니다.

🌟 값 차이(뱀셈)로 찾은 테두리 검출된 테두리: 21개 셀

총합 테두리 ↖️ 오른쪽 셀(세로선) ↘️ 아래쪽 셀(가로선)



경계면에서 [이웃 픽셀 뱀셈]을 수행하여 찾은 테두리입니다.

☒ 초록 = 테두리 픽셀 ☐ 검정 = 배경

🌟 테두리 판정 임계값 (Threshold) 100

▲ 컴퓨터 비전 시뮬레이터 — 8×8 픽셀 캔버스에서 테두리 찾기까지

## START · 이 모듈의 지도

# 4단계를 오르면 여러분도 차량 카운터의 원리를 설명할 수 있습니다

이 모듈에서는 컴퓨터가 '눈'으로 세상을 보는 방식을 배웁니다. 놀랍게도 컴퓨터의 눈에는 사진이 보이지 않습니다. 보이는 것은 **숫자들의 바둑판**뿐이에요. 그 숫자들을 **빨셈**하고 세는 것만으로 테두리를 찾고, 자동차를 셀 수 있습니다.

## 학습 순서 — 4단계

### 1 1단계 · 픽셀·행렬·테두리

8×8 도화지에 그림을 그리고 이웃 빨셈으로 테두리를 찾는 수학 기초

### 2 2단계 · 웹캠 실시간 테두리

내 손과 얼굴의 테두리를 실시간으로 찾아보는 체험

### 3 3단계 · 고속도로 차량 카운터

움직이는 픽셀 덩어리를 세어 자동차·오토바이를 카운트

### 4 4단계 · 보행자 거리 카운터

같은 원리로 사람을 세는 응용

## 이 모듈에서 배우는 것 (3줄 요약)

- ① 컴퓨터의 눈에는 사진이 아니라 **숫자(픽셀)**가 보입니다.
- ② 이웃 숫자끼리 **빨셈**하면 테두리가 보이고, **임계값**으로 잡티를 걸러냅니다.
- ③ 배경과 차이 나는 **픽셀 덩어리**를 세면 자동차·보행자를 자동으로 카운트할 수 있습니다.

### 준비 완료 체크

- 시뮬레이터 페이지와 Colab 노트북(실습)\_물체\_인식.ipynb)을 열어 두었다.
- 2단계 실습을 위해 웹캠 사용을 허락할 준비가 되었다. (없어도 1·3단계는 가능)

## 도입 · 질문

# 컴퓨터의 눈에는 사진이 보이지 않습니다

여러분이 보는 하트  —

컴퓨터는 이것을 **숫자 64개**로 봅니다

8×8 바둑판에 하트를 그렸다고 해 보세요. 여러분의 눈에는 하트 모양이 보이지만, 컴퓨터의 기억 속에는 **0(검정)**과 **255(흰색)** 같은 숫자 64개가 줄지어 있을 뿐입니다. 그런데 이 숫자들만으로 컴퓨터는 하트의 테두리를 찾을 수 있습니다. 어떻게일까요?

## 🔑 핵심 아이디어 미리 보기

테두리란 '숫자가 갑자기 바뀌는 경계'입니다. 하트의 안(255)과 밖(0)이 만나는 곳에서는 이웃 숫자끼리 꽤 큰 차이가 나죠. 이 '차이가 큰 곳 = 테두리'라는 단순한 아이디어가 컴퓨터 비전의 출발점입니다.

## 💡 비유 — 모눈종이

모눈종이에 0과 255를 적어 넣는다고 상상해 보세요. 하트 모양 부분만 255, 나머지는 0으로 채우면 여러분도 '숫자로 된 그림'을 가진 셈입니다. 이제 눈을 감고 숫자만으로 하트의 가장자리를 찾아보세요 — 이웃과 숫자가 다른 칸이 바로 테두리입니다!

### 이 모듈에서 배우는 것 (3줄 요약)

1. 컴퓨터의 눈에는 사진이 아니라 **숫자(픽셀)**가 보여요!
2. 이웃 숫자끼리 **험험**하면 그림의 **테두리**가 보이고, **기준선(임계값)**으로 잡티를 걸러낼 수 있어요!
3. 배경과 **차이** 나는 **픽셀 덩어리**를 세면 **자동차·보행자**를 **자동으로 카운트**할 수 있어요!

준비물: 이 시뮬레이터 페이지 + Colab 노트북 실습)\_물체\_인식.ipynb

선행 지식: 필요 없어요! 뽀빠만 할 수 있으면 충분해요 😊

▲ 모듈 2 도입 — 컴퓨터의 눈은 픽셀(숫자)로 세상을 봅니다

## 개념 ①

## 픽셀과 행렬 — 숫자로 된 그림

## 픽셀(Pixel)

그림을 이루는 가장 작은 바둑판 칸. 'picture element'의 줄임말입니다.

## 행렬(Matrix)

픽셀들을 가로·세로로 정리한 숫자 표. 8×8 그림은 8행 8열의 숫자 표가 됩니다.

## 밝기 값

검은색은 0, 흰색은 255로 기억합니다. 그 사이는 회색입니다.

 행렬로 그림 읽기

시뮬레이터의  컴퓨터의 뇌: 8×8 숫자 행렬 표를 보세요. 캔버스에 하트를 그리면 행렬의 해당 칸이 255로 바뀌며 실시간으로 동기화됩니다. 그림을 그리는 손과 숫자를 보는 눈 — 두 시점을 오가며 '그림 = 숫자'임을 체감하는 것이 이 단계의 목표입니다.

 하트 그리기 버튼을 눌러 보세요.

하트 모양대로 행렬에 255가 채워집니다.

 자동차 도면 버튼도 눌러 보세요.

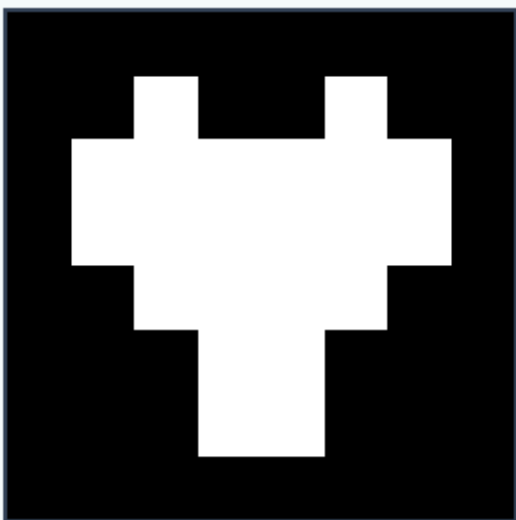
더 복잡한 모양도 결국 숫자의 배열일 뿐입니다.

 교사용 팁 — 수업 운영

학생들에게 '이 행렬을 보고 원래 그림을 맞춰 보라'고 해 보세요. 숫자만으로 그림을 복원하는 과정에서 행렬의 개념이 확실히 자리잡습니다.

 8×8 픽셀 캔버스 (도화지)

지우기



마우스로 칸을 클릭하거나 드래그하여 흰색(255)을 칠해보세요.  
AI Life — 인공지능과 일상

 컴퓨터의 뇌: 8×8 숫자 행렬 (밝기 0~255)

|   |     |     |     |     |
|---|-----|-----|-----|-----|
| 0 | 0   | 0   | 0   | 0   |
| 0 | 0   | 255 | 0   | 0   |
| 0 | 255 | 255 | 255 | 255 |
| 0 | 255 | 255 | 255 | 255 |
| 0 | 0   | 255 | 255 | 255 |
| 0 | 0   | 0   | 255 | 255 |
| 0 | 0   | 0   | 255 | 255 |
| 0 | 0   | 0   | 0   | 0   |

 실시간 행렬 검사기: 위 행렬 표의 칸에 마우스를 올리면 [현재 행렬 계산 과정]이 여기에 표시됩니다.

외부 강의용 교재

## 개념 ②

## 이웃 뿔셈 — 테두리를 찾는 마법의 계산

이제 숫자만으로 테두리를 찾습니다. 방법은 놀라울 만큼 단순합니다. 이웃 칸끼리 빼서 차이가 큰 곳을 테두리라고 부르는 거예요.

## ▶ 뿔셈 위치에 따른 테두리 방향

## 👉 오른쪽 셀 뿔셈

$|오른쪽\ 셀(r, c+1) - 현재\ 셀(r, c)|$  — 좌우 밝기가 바뀔 때 큰 숫자가 나와 세로선(수직 테두리)을 감지합니다.

## 👇 아래쪽 셀 뿔셈

$|아래쪽\ 셀(r+1, c) - 현재\ 셀(r, c)|$  — 상하 밝기가 바뀔 때 큰 숫자가 나와 가로선(수평 테두리)을 감지합니다.

## ✨ 종합 테두리

$\max(오른쪽\ 차이, 아래쪽\ 차이)$  — 세로선과 가로선을 합쳐 완전한 물체 외곽선을 완성합니다.

## 💡 비유 — 담장

밤에 담장을 더듬으며 걷는다고 상상해 보세요. 아무것도 안 닿다가(0) 갑자기 벽이 만져지면(255) '여기가 경계구나' 알게 되죠.



## 개념 ③

## 임계값 — '진짜 테두리'의 기준선

임계값(Threshold)은 "차이가 몇 이상이어야 진짜 테두리로 인정할까?"를 정하는 기준 숫자입니다.

**차이  $\geq$  임계값  $\rightarrow$  테두리 인정**

예: 임계값 100일 때, 이웃 차이가 255면 테두리, 30이면 무시

### 👉 임계값을 낮추면

약한 선도 모두 감지됩니다. 테두리는 굵어지지만 잡티(노이즈)도 함께 보입니다.

### 높게 👉

아주 뚜렷한 선만 남습니다. 잡티는 사라지지만 약한 테두리도 지워질 수 있습니다.

## 🔧 직접 만져보기

시뮬레이터의  테두리 판정 임계값 슬라이더를 좌우로 움직여 보세요. 초록색 테두리 픽셀이 굵어졌다 가늘어지는 것이 실시간으로 보입니다. 이 '기준선 정하기'는 AI의 수많은 판단 과정에 숨어 있는 공통 원리입니다.

### ✅ 1단계 체크포인트 ✅

- 8×8 도화지에 그림을 그리고 👉 오른쪽 셀 뿔셈으로 세로선이 나타나는 것을 확인했다.
- 임계값 슬라이더를 움직여 선이 굵어졌다 가늘어지는 것을 보았다.

## 체험 ① · 시뮬레이터

## 8×8 도화지에 그림을 그리고 테두리 찾기

시뮬레이터의 첫 화면은 8×8 픽셀 캔버스(도화지)입니다. 마우스로 칸을 클릭하거나 드래그하면 흰색 (255)이 칠해지고, 오른쪽의 숫자 행렬과 테두리 화면이 실시간으로 바뀝니다.

## 1 그림 그리기

마우스로 드래그하거나 ♥ 하트 · 🚗 자동차 도면 · — 대각선 · □ 사각 상자 프리셋을 눌러 그림을 그립니다.

## 2 숫자 확인

 컴퓨터의 뇌 행렬 표에서 그린 모양이 0과 255의 숫자로 바뀌었는지 확인합니다.

## 3 테두리 관찰

✨ 값 차이로 찾은 테두리 화면에서 초록색 테두리 픽셀이 나타나는지 봅니다. (초록=테두리, 검정=배경)

## 4 버튼 실험

✨ 종합 테두리 / 👉 오른쪽 셀(세로선) / 👇 아래쪽 셀(가로선) 버튼을 눌러 방향별 테두리를 비교합니다.

## 🔍 실시간 뱀셈 검사기 활용법

행렬 표의 칸에 마우스를 올리면 [현재 셀] - [👉 오른쪽 셀], [현재 셀] - [👇 아래쪽 셀]의 계산 과정이 그대로 표시됩니다. 테두리 경계에 있는 칸과 한가운데 있는 칸을 번갈아 올려놓고 차이 값을 비교해 보세요. 숫자가 직접 말해 줍니다 — "여기가 경계야!"

## ✅ 1단계 체크포인트 ✅

- 도화지에 그림을 그리고 👉 오른쪽 셀 뱀셈 버튼을 눌렀을 때 세로선이 나타나는 것을 확인했다.
- 임계값 슬라이더를 움직여 선이 굵어졌다 가늘어지는 것을 보았다.

## 🍷 교사용 팁 — 수업 운영

2명이 짝을 지어 한 명은 눈을 감고, 다른 한 명은 행렬 숫자만 읽어주는 게임을 해 보세요. 숫자만으로 그림을 맞히는 과정에서 '컴퓨터의 시각'을 몸으로 이해하게 됩니다.

## 체험 ② · 시뮬레이터

## 웹캠으로 내 손의 테두리 찾기

1단계의 원리를 실제 카메라에 적용합니다. 매 프레임마다 모든 픽셀의 가로·세로 이웃 뽀렘(Sobel/Canny 연산)을 수행하면, 내 손과 얼굴의 테두리가 실시간으로 그려집니다.

### 1 웹캠 켜기

▶ 웹캠 켜기 버튼을 누르고 브라우저의 카메라 권한을 허용합니다.

### 2 듀얼 뷰 관찰

왼쪽은 원본(RGB), 오른쪽은 뽀렘으로 찾은 테두리 화면이 나란히 표시됩니다.

### 3 임계값 조절

☛ 테두리 임계값 슬라이더를 움직이며 테두리의 변화를 관찰합니다.

### 4 색상 변경

테두리 선 색상을 에메랄드 그린·테크 블루·흰색으로 바꿔 볼 수 있습니다.

### 🔍 관찰 포인트

**임계값을 낮추면:** 배경의 잔주름이나 옷깃의 미세한 선까지 모두 나타납니다.

**임계값을 높이면:** 손과 몸의 가장 선명한 외곽 테두리만 남습니다.

같은 장면, 같은 계산인데 기준선 하나 바꿨을 뿐인데 결과가 완전히 달라집니다. 이것이 **파라미터의 힘**입니다.

### ✅ 2단계 체크포인트 ✅

- 임계값을 낮추면 잔주름까지 보이고, 높이면 외곽선만 남는 것을 확인했다.

## 체험 ③ · 시뮬레이터

## 고속도로 차량 카운터 — 픽셀 덩어리 세기

이제 테두리 찾기를 응용합니다. 고속도로 영상에서 지나가는 자동차를 자동으로 세는 것이 목표입니다. 핵심 원리는 **배경 차분** — 멈춰 있는 배경을 미리 기억해 두고 현재 화면에서 빼면, 움직인 물체만 흰색 덩어리로 남습니다.

## 1 이중 화면 관찰

👁️ 실제 화면(초록 인식 상자)과 🤖 컴퓨터 시각(흑백 덩어리)을 비교합니다.

## 2 통과선 통과

물체가 노란색 통과선을 지나면 통과 기록표에 ID·종류·시각·크기·속도가 기록됩니다.

## 3 최소 크기 조절

📏 슬라이더를 작게 하면 오토바이까지, 크게 하면 자동차만 셉니다.

## 4 기록 저장

💾 기록 엑셀(CSV) 저장 버튼으로 데이터를 내려받을 수 있습니다.

## ★ 크기 필터링의 원리

흰색 덩어리의 넓이가 설정한 최소 크기(min\_area)보다 클 때만 '진짜 물체'로 인정합니다. 오토바이는 작고 자동차는 크니까, 기준선 하나로 둘을 구분할 수 있는 거예요. 1단계의 임계값과 똑같은 '기준선 정하기' 원리가 여기서도 등장합니다!

## ✓ 3·4단계 체크포인트 ✓

- '최소 크기'를 작게 했을 때 오토바이까지 세어지고, 크게 했을 때 자동차만 세어지는 것을 확인했다.

AI Life 홈

02 컴퓨터비전

컴퓨터의 눈: 픽셀·행렬·테두리부터 물체 인식까지

Colab 실행

1단계: 픽셀·행렬·테두리 수학 기초

2단계: 웹캠 실시간 테두리 & 임계값

3단계: 고속도로 차량/오토바이 카운터

4단계: 보행자 거리 카운터

도로 모드: 크기 슬라이더를 작게 하면 오토바이도 세고, 크게 하면 자동차만 셉니다.

고속도로 차량 & 오토바이 통과 모니터링

일시 정지

카운트 0으로

오토바이 1대 추가

TOTAL OBJECT COUNT

노란색 통과선을 통과한 총 차량 대수

통과: 30 대

01. 실제 화면 (초록색 인식 상자)

02. 컴퓨터 시각 (배경과 차이 난 흑백 덩어리)

초록 박스: 인식된 물체

노란선: 카운트 통과선

흰 덩어리: 움직인 픽셀

물체 인식 조절기 (학생용 쉬운 설정)

기본값으로

감지할 물체의 최소 크기

100 px

[오토바이 & 사람까지 모두 인식 중]

작게 (오토바이 포함)

크게 (자동차만)

작게 (오토바이 감지)

크게 (자동차만 감지)

카운트 통과선(노란선) 높이

60%

카메라 탐색 구역 (위 / 아래 범위)

20% ~ 85%

위:

아래:

조각난 흰색 점 뭉쳐주기 강도

외부 강의용 교재

▲ 고속도로 차량 & 오토바이 통과 모니터링 — 노란 통과선을 지나는 물체를 카운트합니다

## 코드 ① · 파이썬

## 체험을 코드로 — 이웃 뿔셈 테두리 검출

시뮬레이터가 화면 뒤에서 하던 일을 파이썬으로 옮기면 아래와 같습니다. Colab 노트북 '(실습)\_물체\_인식.ipynb'의 셀들과 같은 내용이니 한 줄씩 실행하며 비교해 보세요.

```
# 1. 픽셀 이웃 뿔셈을 통한 테두리(Edge) 찾기
# - 1단계 시뮬레이터에서 마우스로 직접 해본 '오른쪽 셀/아래쪽 셀 뿔셈'이 바로 이 함수예요.
def detect_edges_by_difference(gray_image, threshold=30):
    # 가로(오른쪽 셀) 뿔셈: 세로선 감지
    diff_right = np.abs(gray_image[:, 1:].astype(int)
                        - gray_image[:, :-1].astype(int))
    # 세로(아래쪽 셀) 뿔셈: 가로선 감지
    diff_down = np.abs(gray_image[1:, :].astype(int)
                      - gray_image[:-1, :].astype(int))

    # 임계값(Threshold) 비교
    edge_right = (diff_right >= threshold) * 255
    edge_down = (diff_down >= threshold) * 255
    return edge_right.astype(np.uint8), edge_down.astype(np.uint8)
```

 (실습)\_물체\_인식.ipynb — 테두리 검출 셀

### 코드 읽기 포인트

- ① **gray\_image[:, 1:] - gray\_image[:, :-1]**: 전체 이미지의 오른쪽 뿔셈을 한 줄로! NumPy는 행렬 전체를 한 번에 계산하므로, 8×8이든 1920×1080이든 같은 코드로 처리됩니다.
- ② **np.abs(...)**: 절댓값 — 밝아지든 어두워지든 '변화의 크기'만 봅니다.
- ③ **>= threshold**: 시뮬레이터의 슬라이더가 바로 이 숫자입니다.

## 코드 ② · 파이썬

## 배경 차분(MOG2)으로 움직이는 물체만 골라내기

3단계 카운터의 핵심은 **배경 차분**입니다. '움직이지 않는 배경'을 AI가 미리 외워 두고, 현재 화면에서 빼면 움직인 물체만 남는 원리죠.

```
# 2. 고속도로 영상 움직임 감지 & 차량/오토바이 선 통과 카운터
def process_traffic_video(video_path="car_highway2.mp4", min_area=350, line_ratio=0.65):
    cap = cv2.VideoCapture(video_path)
    # 배경 차분기: 배경과 다른 픽셀만 흰색으로 표시해주는 도구
    fgbg = cv2.createBackgroundSubtractorMOG2(history=500,
                                              varThreshold=50, detectShadows=False)
    ...
    fgmask = fgbg.apply(frame) # 움직인 부분만 흰색인 마스크
    contours, _ = cv2.findContours(fgmask, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)

    for cnt in contours:
        area = cv2.contourArea(cnt)
        # ★ 크기 필터링: min_area보다 클 때만 '진짜 물체'로 인정!
        # (작게: 오토바이 포함 / 크게: 자동차만)
        if area >= min_area:
            x, y, cw, ch = cv2.boundingRect(cnt) # 초록 상자 그리기
```

🐼 (실습)\_물체\_인식.ipynb — 카운터 셀 (발체)

## 🔍 파이프라인 한눈에

- 1 **배경 학습**  
MOG2가 수백 프레임의 배경을 기억합니다. (history=500)
- 2 **차분**  
현재 프레임 - 배경 = 움직인 픽셀만 흰색 마스크
- 3 **덩어리 찾기**  
findContours로 흰 덩어리들의 외곽선을 찾습니다.
- 4 **크기 필터**  
min\_area 이상만 통과 — 오토바이/자동차 구분선!
- 5 **통과 판정**  
덩어리 중심이 노란 통과선을 지나면 카운트 +1

## 수학 심화

## 왜 뽀샵일까? — 미분과 소벨 필터

1단계에서 배운 '이웃 뽀샵'은 사실 고등학교 수학의 **미분**과 같은 아이디어입니다. 미분이 '순간 변화율'을 재는 것처럼, 이웃 뽀샵은 '공간 변화율'을 재는 거예요.



### 변화가 크면 = 테두리

함수 그래프에서 기울기가 급한 곳이 '급변하는 구간'이듯, 이미지에서 이웃 차이가 큰 곳이 '급변하는 경계 = 테두리'입니다. 수학자들은 이 아이디어를 정교하게 다듬어 **소벨(Sobel) 필터**와 **캐니(Canny) 에지 검출기**를 만들었습니다. 시뮬레이터 2단계의 웹캠 테두리가 바로 이 연산들을 실시간으로 수행한 결과입니다.

| 방법           | 하는 일                | 특징            |
|--------------|---------------------|---------------|
| 이웃 뽀샵 (이 모듈) | 이웃 - 현재  로 변화 측정    | 가장 단순, 직관적    |
| 소벨 필터        | 3×3 가중치로 부드럽게 변화 측정 | 노이즈에 조금 더 강함  |
| 캐니 검출기       | 여러 단계로 얇고 정확한 선 추출  | 실무 표준, 계산량 많음 |

중요한 것은 세 방법의 출발점이 모두 '**변화 재기**'라는 점입니다. 여러분은 오늘 그 출발점을 손으로 직접 체험했습니다.



### 비유 — 체온계

열이 나는지 알려면 '평소 체온과의 차이'를 재죠. 테두리도 마찬가지입니다. '평소(이웃)와 얼마나 다른가'를 재는 것 — 측정의 기본은 언제나 **비교**입니다.

## 심화 탐구

## 파라미터 실험실 — 기준선을 움직여라

이 모듈에 등장한 두 개의 '기준선'을 직접 실험해 봅시다. 작은 숫자 하나가 AI의 눈을 어떻게 바꾸는지 관찰하는 것이 목표입니다.

### 실험 1 — 임계값(Threshold) sweep

1단계 캔버스에 하트를 그리고 임계값을 10 → 50 → 100 → 200으로 단계별로 올려 보세요. 각 단계에서 테두리 개수를 기록하면, '기준선이 올라갈수록 테두리가 얇아진다'는 그래프를 그릴 수 있습니다.

### 실험 2 — 최소 크기(min\_area) sweep

3단계 카운터에서 최소 크기를 크게 → 작게로 움직이며 '오토바이가 잡히기 시작하는 지점'을 찾아보세요. 그 경계 값이 바로 오토바이와 자동차를 가르는 AI의 판단 기준선입니다.

### 실험 3 — 조각난 점 뭉쳐주기

조절기의  조각난 흰색 점 뭉쳐주기 강도를 바꿔 보세요. 이것은 흩어진 픽셀을 하나로 묶는 '모폴로지 연산'의 강도로, 너무 약하면 한 대의 차가 여러 개로 쪼개지고 너무 강하면 옆 차와 합쳐져 버립니다. '적당한 뭉침'을 찾는 것도 AI 조정의 일부입니다.

#### 교사용 팁 — 토론 주제

CCTV가 사람을 셀 때 '최소 크기'를 잘못 정하면 어떤 일이 생길까요? 어린이가 무시되거나, 강아지가 사람으로 카운트될 수 있습니다. 파라미터 하나가 현실 세계에 미치는 영향을 토론해 보세요.

## 세상 속 AI

## 우리 주변의 컴퓨터 비전

오늘 배운 '빨셈 → 테두리 → 덩어리 세기' 파이프라인은 교실 밖 세상을 이미 바꾸고 있습니다.

### 자율주행 자동차

차선(이웃 차이로 찾은 테두리!), 앞차(움직이는 덩어리), 신호등을 실시간으로 인식합니다. 여러분이 2단계에서 본 웹캠 테두리 화면과 본질적으로 같은 기술을 씁니다.

### 공장 불량 검출

컨베이어벨트 위 제품의 테두리가 기준과 다르면 불량으로 판정합니다. 사람 눈보다 빠르고 지치지 않죠. '정상 제품의 숫자 행렬'을 외워 두고 비교하는 방식입니다.

### 얼굴 인식 잠금해제

얼굴의 눈·코·입 테두리 패턴을 숫자로 바꿔 저장해 둡니다. 잠금해제 때 얼굴을 비추면 패턴을 비교하죠. 다음 모듈(CNN)에서 이 기술의 심화판을 배웁니다.

### 무인 매장

선반에서 상품이 사라진 것(배경과 달라진 덩어리)을 감지해 자동으로 계산합니다. 3단계 차량 카운터와 정확히 같은 '배경 차분' 원리입니다.

 윤리적 질문 — 보는 AI, 감시하는 AI: 같은 기술이 교통량 조사에도, 무단 감시에도 쓰일 수 있습니다. 기술을 배울 때는 항상 '이걸 어디에 쓰면 좋을까?'를 함께 고민해야 합니다.

## Q&amp;A

## 자주 묻는 질문 — 한 걸음 더 깊이

### ? 사진 속 색깔(빨강·파랑)은 어떻게 숫자가 되나요?

컬러 이미지는 빨강(R)·초록(G)·파랑(B) 3장의 숫자 행렬을 겹친 것입니다. 각 픽셀이 (R, G, B) 세 숫자를 갖죠. 시뮬레이터에서는 이해를 돕기 위해 흑백(밝기 0~255) 하나로 단순화했습니다. 원리는 같습니다 — 3배의 숫자를 다룰 뿐이에요.

### ? 테두리만 찾으면 물체를 '알아보는' 건가요?

아직 아닙니다! 테두리 찾기는 '어디에 뭔가 있다'를 아는 단계이고, 그것이 자동차인지 사람인지를 아는 것은 다음 단계의 일입니다. 3단계 카운터에서는 크기·움직임 같은 단서로 '자동차 같음'을 추측하죠. 진짜로 '이건 자동차야'라고 알아보는 기술이 다음 모듈의 CNN입니다.

### ? 배경 차분은 카메라가 흔들리면 어떻게 되나요?

예리한 지적입니다! 카메라가 흔들리면 배경 전체가 '움직인 것'으로 보여 엉망이 됩니다. 실제 CCTV 카운터는 카메라를 단단히 고정하고, 그림자 제거(detectShadows) 같은 보정을 더합니다. 완벽한 기술은 없고, **조건에 맞는 설계**가 있다는 것을 보여주는 좋은 예입니다.

### ? AI가 직접 테두리를 '학습'할 수는 없나요?

있습니다 — 그게 바로 다음 모듈 CNN입니다. 사람이 '빨셈하라'고 알려주지 않아도, AI가 수많은 사진을 보고 스스로 테두리 찾는 필터를 만들어냅니다. 손수레에서 자동차로 넘어가는 순간을 기대하세요!

## 정리 · 체크리스트

여기까지 되면  — 모듈 2 완주!

컴퓨터의 눈이 되는 여정이 끝났습니다. 아래를 확인해 보세요.

 핵심 개념 체크

- 컴퓨터에게 그림은 **픽셀의 행렬(0~255 숫자 표)**임을 설명할 수 있다.
- 이웃 **뽕셈**으로 테두리를 찾는 원리(오른쪽=세로선, 아래쪽=가로선)를 설명할 수 있다.
- **임계값**이 '진짜 테두리'의 기준선임을 알고, 조절 효과를 설명할 수 있다.
- **배경 차분**으로 움직인 물체만 골라내는 원리를 설명할 수 있다.
- **최소 크기 필터**로 오토바이와 자동차를 구분하는 아이디어를 안다.

 체험·코드 체크

- 8×8 캔버스에 그림을 그리고 테두리가 나타나는 것을 보았다.
- 웹캠 테두리에서 임계값에 따른 변화를 관찰했다.
- 차량 카운터에서 최소 크기를 바꿔 오토바이 포함/제외를 확인했다.
- `detect_edges_by_difference` 함수의 3줄 핵심을 읽을 수 있다.

 한 문장 정리

컴퓨터 비전 = 그림을 숫자 행렬로 보고, 이웃 뽕셈으로 테두리를 찾고, 기준선(임계값·최소 크기)으로 의미를 골라내는 것.

 교사용 팁 — 다음 차시 예고

다음 모듈에서는 오늘의 '손으로 만든 테두리 찾기'를 AI가 스스로 학습하는 버전 — **CNN 합성곱 신경망**을 배웁니다. 6×6 픽셀에 손글씨를 그려 보는 시간입니다.